

**Centro Universitario de Ciencias Exactas e
Ingenierías**



Materia: Interacción Humano Computadora

Actividad: Hands-On 4: Actividad Integradora

Alumno: Alvarado Lomeli Gael Ramses

Profesor: Jose Antonio Aviña Méndez

Carrera: Ingeniería en Computación (ICOM)

Fecha: 24 de mayo del 2026

índice

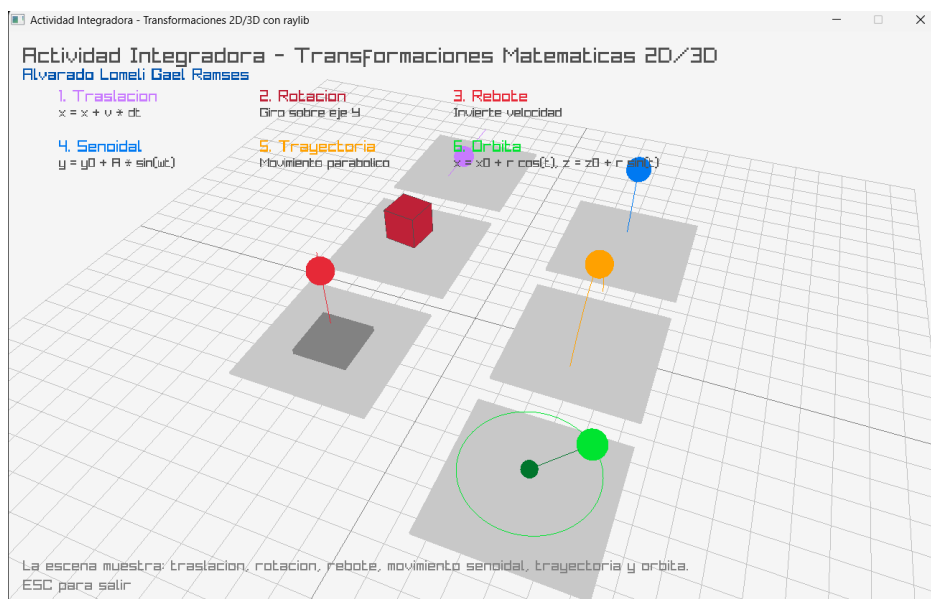
Introducción.....	3
Transformaciones.....	4
1. Traslación.....	4
2. Rotación.....	4
3. Rebote.....	5
4. Movimiento Senoidal.....	7
5. Trayectoria parabólica.....	8
6. Orbita.....	9
Conclusiones.....	10
Referencias bibliográficas.....	11

Introducción

En esta actividad integradora desarrollé una aplicación gráfica utilizando el lenguaje C++ junto con la biblioteca raylib. El propósito principal del programa fue representar visualmente distintas transformaciones y movimientos matemáticos estudiados en clase, aplicados en un entorno tridimensional. Las transformaciones implementadas fueron traslación, rotación, rebote, movimiento senoidal, trayectoria parabólica y órbita.

La biblioteca raylib permite trabajar con gráficos 2D y 3D mediante funciones sencillas para crear ventanas, dibujar figuras, controlar cámaras y actualizar animaciones en tiempo real. Su diseño está orientado a la programación gráfica de forma directa, lo cual facilita la comprensión de conceptos visuales como posición, velocidad, rotación y movimiento en el espacio. Además, raylib utiliza internamente OpenGL para el renderizado acelerado por hardware y proporciona funciones como DrawCube, DrawSphere, DrawLine3D, BeginMode3D y UpdateCamera, las cuales fueron utilizadas en el desarrollo de esta actividad.

El programa fue estructurado en una sola escena 3D dividida en seis zonas. Cada zona representa un tipo de transformación o movimiento distinto. Esta organización permite comparar visualmente los comportamientos matemáticos de cada objeto dentro de una misma ventana. Para lograr animaciones independientes del rendimiento del equipo, se utilizó `GetFrameTime()`, lo cual permite actualizar las posiciones en función del tiempo transcurrido entre fotogramas y no únicamente por la cantidad de ciclos de ejecución.



Transformaciones

1. Traslación

La traslación es una transformación geométrica que desplaza un objeto desde una posición inicial hacia otra posición, sin modificar su forma, tamaño ni orientación. En el programa, esta transformación se representa mediante una esfera morada que se mueve horizontalmente sobre el eje X dentro de su zona correspondiente.

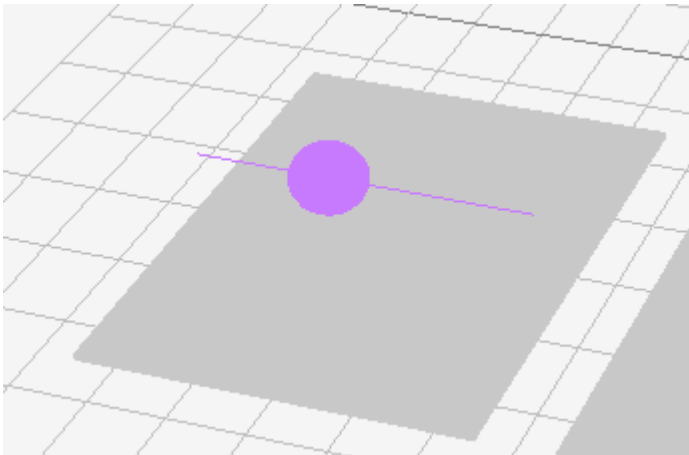
Matemáticamente, la posición del objeto se actualiza sumando el producto de la velocidad por el tiempo transcurrido:

$$x = x + v \cdot dt$$

En el código, esta transformación se implementa con la variable `trasX`, la cual aumenta con respecto al tiempo:

```
trasX += velocidadTras * dt;
```

Cuando la esfera supera el límite establecido, su posición se reinicia para volver a comenzar el recorrido. De esta forma, se observa un desplazamiento lineal continuo. Esta representación permite comprender que la traslación afecta directamente la posición del objeto, pero no altera sus dimensiones ni su orientación.



2. Rotación

La rotación es una transformación en la que un objeto gira alrededor de un eje. En el programa, la rotación se representa mediante un cubo color marrón que gira

sobre su propio eje Y. Para lograr este efecto, se utilizaron las funciones `glPushMatrix()`, `glTranslatef()` y `glRotatef()`.

En términos matemáticos, una rotación en dos dimensiones puede expresarse mediante las siguientes ecuaciones:

$$x' = x \cos(\theta) - y \sin(\theta)$$

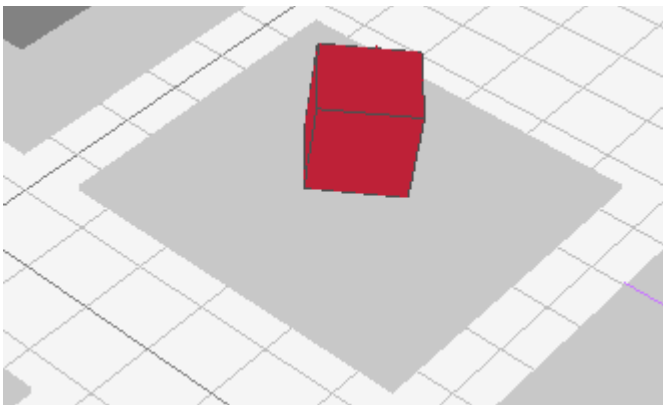
$$y' = x \sin(\theta) + y \cos(\theta)$$

Estas ecuaciones muestran cómo las coordenadas de un punto cambian cuando se aplica un ángulo de rotación θ . En el caso del programa, la rotación se aplica en un entorno tridimensional alrededor del eje Y, por lo que el objeto mantiene su centro en una posición fija mientras cambia su orientación.

El fragmento principal de código es el siguiente:

```
glTranslatef(-3.0f, 1.2f, 4.0f);
glRotatef(angulo, 0.0f, 1.0f, 0.0f);
```

El uso de matrices de transformación es una base fundamental en gráficos por computadora. OpenGL describe la traslación, rotación y escalamiento como transformaciones equivalentes a multiplicaciones matriciales aplicadas sobre los vértices de los objetos.



3. Rebote

El rebote se implementó como un movimiento vertical en el que una esfera roja sube y baja entre dos límites definidos. Cuando la esfera alcanza el límite superior

o inferior, la dirección de su velocidad se invierte. Esta inversión representa el cambio de dirección producido por el contacto con una frontera.

Matemáticamente, la actualización de la posición se expresa como:

$$y = y + v_y \cdot dt$$

Cuando el objeto llega a un límite, se aplica:

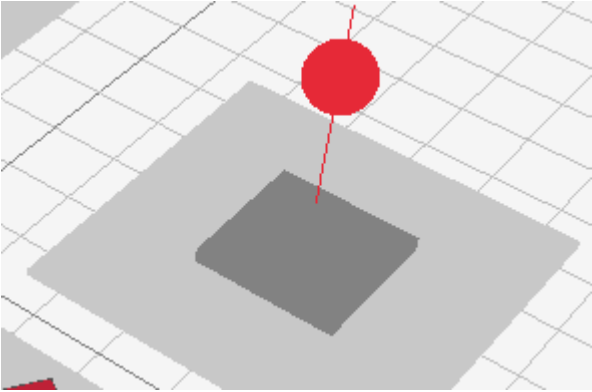
$$v_y = -v_y$$

En el código, esto se observa en la condición que evalúa si la esfera llegó a los límites:

```
if (reboteY > 4.0f)
{
    reboteY = 4.0f;
    velocidadRebote *= -1.0f;
}

if (reboteY < 1.0f)
{
    reboteY = 1.0f;
    velocidadRebote *= -1.0f;
}
```

Este comportamiento permite visualizar la relación entre posición, velocidad y cambio de dirección. El rebote no modifica la forma del objeto, sino solamente el sentido de su movimiento.



4. Movimiento senoidal

El movimiento senoidal se basa en la función seno, la cual produce una oscilación suave y periódica. En el programa, este movimiento se representa mediante una esfera azul que sube y baja de manera continua sobre el eje Y.

La ecuación utilizada para este movimiento es:

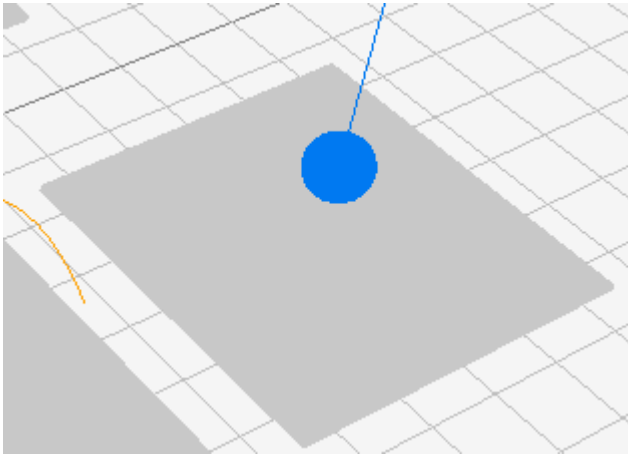
$$y = y_0 + A \sin(\omega t)$$

Donde y_0 representa la posición central, A es la amplitud, ω es la frecuencia angular y t es el tiempo. Este tipo de movimiento está relacionado con oscilaciones periódicas y con el movimiento armónico simple, donde la posición puede describirse mediante funciones trigonométricas. OpenStax presenta el movimiento circular y oscilatorio mediante componentes basadas en seno y coseno, relacionando la posición con el radio, el tiempo y la frecuencia angular.

En el código, la posición vertical se calcula así:

```
float senoY = 2.3f + sinf(tiempo * 2.0f) * 1.3f;
```

Esta expresión indica que la esfera oscila alrededor de la altura 2.3, con una amplitud de 1.3, y con una frecuencia controlada por el factor 2.0. La función `sinf()` permite calcular el valor del seno en cada instante de tiempo.



5. Trayectoria parabólica

La trayectoria representa el movimiento de un objeto siguiendo una curva. En el programa, esta transformación se muestra mediante una esfera naranja que se desplaza siguiendo una curva parabólica. Esta trayectoria se relaciona con el movimiento de proyectiles, en el que el desplazamiento horizontal y el desplazamiento vertical se analizan de manera independiente.

La posición horizontal se calcula como un movimiento aproximadamente uniforme:

$$x = x_0 + v_x t$$

La posición vertical se calcula mediante una ecuación cuadrática:

$$y = y_0 + v_y t - \frac{1}{2} g t^2$$

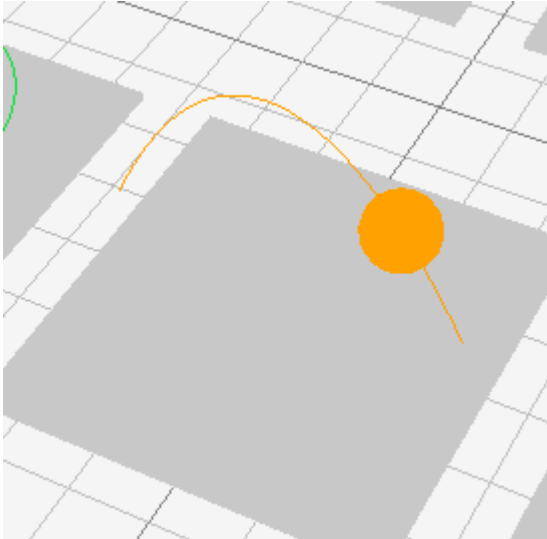
En el programa, se utilizó una forma adaptada de esta ecuación:

```
float trayX = -2.2f + tiempoTrayectoria * 1.6f;

float trayY = 0.8f + 3.8f * tiempoTrayectoria - 1.35f *
tiempoTrayectoria * tiempoTrayectoria;
```

El término cuadrático negativo provoca que la esfera suba inicialmente y después descienda, formando una parábola. Este comportamiento coincide con el modelo físico del movimiento de proyectiles, donde la aceleración horizontal es cero y la aceleración vertical corresponde a la gravedad. OpenStax explica que el movimiento de proyectiles puede separarse en un movimiento horizontal con velocidad constante y un movimiento vertical con aceleración constante.

Además, el programa dibuja una línea naranja que indica la trayectoria prevista. Esto permite observar no solo la posición instantánea del objeto, sino también el camino completo que sigue durante su movimiento.



6. Órbita

La órbita es un movimiento circular alrededor de un punto central. En el programa, este movimiento se representa mediante una esfera verde que gira alrededor de un centro fijo. Para calcular su posición se utilizaron las funciones trigonométricas coseno y seno, aplicadas sobre los ejes X y Z.

La posición orbital se calcula mediante las siguientes ecuaciones:

$$x = x_0 + r \cos(t)$$

$$z = z_0 + r \sin(t)$$

Donde x_0 y z_0 son las coordenadas del centro de la órbita, r es el radio y t representa el tiempo. En el código, esto se implementa así:

```
float orbitaX = cosf(tiempo) * radioOrbita;
```

```
float orbitaZ = sinf(tiempo) * radioOrbita;
```

La esfera se dibuja en una posición que depende de esos valores:

```
DrawSphere(
    { 6.0f + orbitaX, 1.0f, -4.0f + orbitaZ },
    0.45f,
```

GREEN

);

El movimiento circular uniforme puede representarse mediante un vector de posición con componentes trigonométricas. OpenStax expresa este tipo de movimiento con componentes basadas en $A \cos(\omega t)$ y $A \sin(\omega t)$, donde A representa el radio de la trayectoria circular.

Conclusiones

El desarrollo de esta aplicación permitió integrar conceptos matemáticos, físicos y computacionales dentro de una misma escena gráfica. A través de raylib y C++, fue posible representar transformaciones fundamentales como la traslación y la rotación, además de movimientos dinámicos como el rebote, el movimiento senoidal, la trayectoria parabólica y la órbita.

La actividad mostró que las transformaciones gráficas no son únicamente efectos visuales, sino aplicaciones directas de modelos matemáticos. Cada movimiento observado en pantalla responde a una ecuación específica que determina la posición del objeto en función del tiempo. La traslación utiliza una relación lineal entre posición y velocidad; la rotación depende del cambio angular; el rebote invierte la velocidad al alcanzar límites; el movimiento senoidal utiliza una función periódica; la trayectoria parabólica combina movimiento horizontal y vertical; y la órbita emplea funciones trigonométricas para generar movimiento circular.

También fue importante utilizar `GetFrameTime()`, ya que esto permitió que la animación dependiera del tiempo real y no únicamente de la velocidad de procesamiento del equipo. Esto mejora la estabilidad del programa y hace que el movimiento sea más correcto desde el punto de vista computacional.

En conclusión, el programa cumple con el objetivo de mostrar de manera visual las transformaciones matemáticas solicitadas. Además, la implementación fortalece la comprensión de cómo las ecuaciones matemáticas pueden convertirse en animaciones interactivas dentro de un entorno gráfico tridimensional.

Referencias bibliográficas

Hearn, D., Baker, M. P., & Carithers, W. R. (2011). *Computer graphics with OpenGL* (4th ed.). Pearson.

Khronos OpenGL ARB Working Group. (2004). *OpenGL programming guide*. Addison-Wesley.

OpenStax. (2016). *University physics volume 1*. Rice University.

OpenStax. (2022). *College physics 2e*. Rice University.

Santamaría, R. (2026). *raylib: A simple and easy-to-use library to enjoy videogames programming [Computer software]*. GitHub.

Santamaría, R. (2025). *raylib cheatsheet [Documentation]*. raylib.